



TOR VERGATA  
UNIVERSITÀ DEGLI STUDI DI ROMA

# Sky Analytics Engine

Analisi batch e Metriche del Traffico Aereo

---

Flavio Simonelli   Francesco Masci

Corso Architettura e Sistemi per Big Data  
Corso di Laurea Magistrale in Ingegneria Informatica  
Università di Roma "Tor Vergata"

12 Giugno 2026

Architettura e Flusso

Extraction e Ingestion Layer

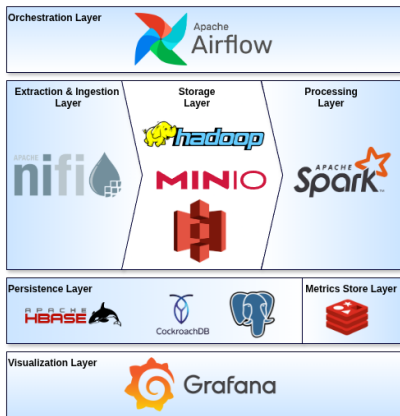
Computation Layer

Orchestration Layer

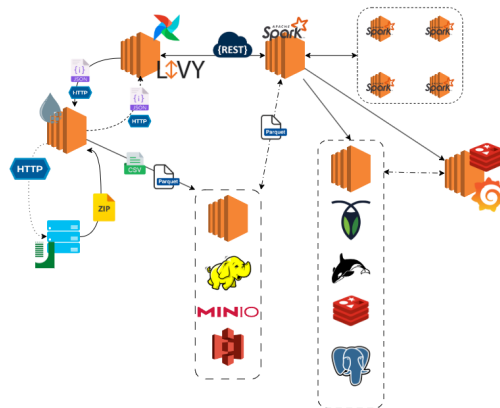
Valutazione Sperimentale

Risultati Analitici

# Big Data Stack & Flow Diagram



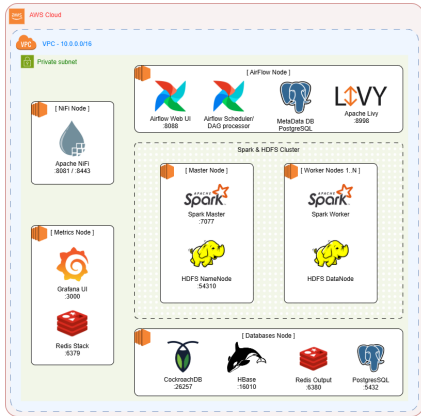
Componenti del Big Data Stack



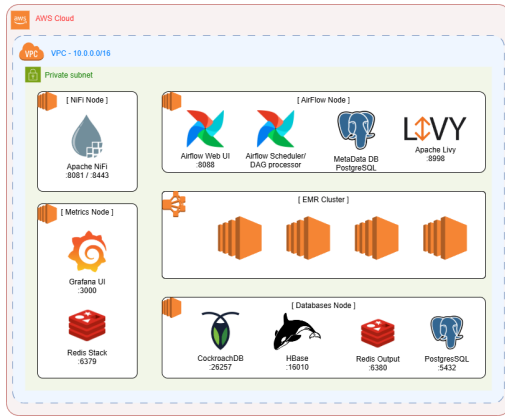
Flusso di interazione dei componenti

# Deployment

## EC2 Deployment



## EMR Deployment

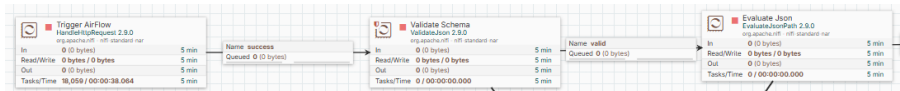


# NiFi Ingestion Pipeline: REST Trigger

- **Design Data-Driven:** Pipeline stateless configurata dinamicamente in base al payload JSON iniettato all'avvio da Airflow.
- **Avvio della Pipeline:**
  - **HandleHttpRequest:** espone l'endpoint REST per intercettare l'avvio.
  - **ValidateJson:** esegue il controllo strutturale rigoroso tramite JSON Schema.
- **Propagazione della Configurazione:**
  - **EvaluateJsonPath:** estrae e mappa le chiavi JSON in Attributi del FlowFile.
  - I metadati guidano le decisioni successive del processing.

## Payload JSON

```
{
  "job_id": "FLIGHT_INGEST_run_123",
  "ingestion": {
    "source_type": "HTTP",
    "http_url": "http://www.ce.uniroma2.it/courses/sabd2526/...",
    "expected_sha1": "17be276b72bd987e72598b0ea4907c3b19350606"
  },
  "storage_raw": { "type": "HDFS", "path": "/data/raw" },
  "storage_preprocessed": {
    "type": "S3",
    "bucket": "spark-flight-analysis",
    "path": "data/conv"
  },
  "processing": { "optimization_strategy": "PARTITION_PRUNING" },
  "callback": {
    "run_id": "run_123",
    "variable_key": "nifi_done_run_123",
    "url": "http://.../variables/nifi_done_run_123",
    "method": "PATCH",
    "headers": {
      "Authorization": "Bearer eyJhbG...",
      "Content-Type": "application/json"
    }
  }
}
```



# NiFi Ingestion Pipeline: Download, Decompressione e Routing

- **Estrazione e Validazione:**

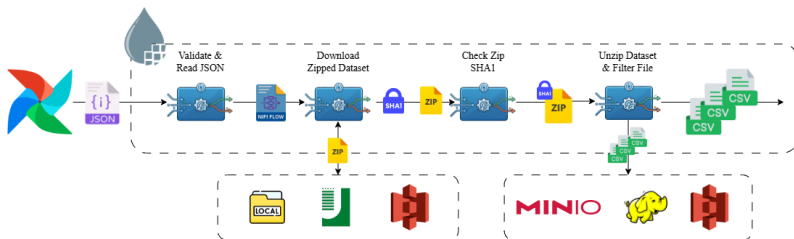
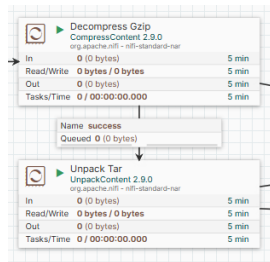
- Download automatico del dataset compresso dall'origine.
- Controllo di integrità tramite `CryptographicHashContent`.

- **Decompressione e Filtraggio:**

- Spacchettamento via `CompressContent` e `UnpackContent`.
- Ispezione e routing selettivo dei soli CSV d'interesse.

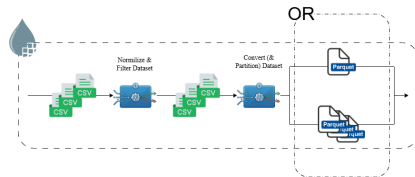
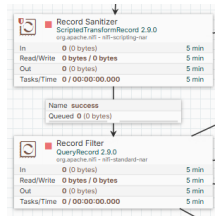
- **Routing dei File:**

- **Ramo Raw:** Salvataggio as-is per garanzia del dato originale.
- **Ramo Preprocessing:** Inoltro dei file d'interesse alla trasformazione.



# NiFi Ingestion Pipeline: Preprocessing

- **Sanitizzazione** - ScriptedTransformRecord:
  - Se esiste una causa di ritardo positiva, imposta le altre nulle a 0.
  - Normalizzazione a 0 delle componenti di ritardo negative.
- **Data Cleaning** - QueryRecord:
  - Taglio dei record con chiavi primarie nulle o dati temporali mancanti.
  - Eliminazione dei voli cancellati con orario di partenza presente.
  - Scarto dei record con delay negativo e componenti di ritardo positive.
- **Strategie di Conversione:**
  - **Partition Pruning:** PartitionRecord converte in Parquet partizionando per chiavi temporali e carrier.
  - **Predicate Pushdown:** MergeRecord compatta e converte in un singolo file Parquet.

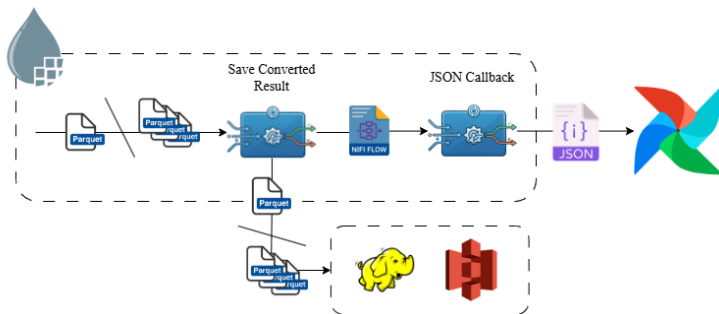


# NiFi Ingestion Pipeline: Storage Finale e Callback ad Airflow

- **Storage Distribuito:** Scrittura dei file Parquet ottimizzati in HDFS o S3.
- **Notifica di Completamento:**
  - Chiamata HTTP PATCH via InvokeHTTP alle API di Airflow.
  - L'url di callback viene fornito nel payload di attivazione.

## Payload JSON

```
{  
  "key": "${airflow.callback.variable_key}",  
  "value": "true"  
}
```





# Configurazione a Start-Time & Design Pattern Java

- **Parametrizzazione a Start-Time:**

- **Configurazione senza ricompilazione:** Gestione via YAML (`ApplicationConfig`) con override da **CLI**.
- **Selezione del Backend:** Scelta a runtime del paradigma di esecuzione (*RDD*, *DataFrame*, *SQL*), del partizionamento e delle strategie di calcolo (KLL vs T-Digest).
- **Storage Agnostic:** Disaccoppiamento totale dell'I/O su HDFS, S3, Local FS, PostgreSQL, CockroachDB, Redis e HBase.

- **Design Pattern Java:**

- **Template Method:** `BaseQuery.execute()` standardizza e unifica l'intero ciclo di vita del job Spark.
- **Factory & Repository:** `FlightRepositoryFactory` gestisce il polimorfismo, astruendo le connessioni ai database.
- **Strategy:** Switch dinamico degli algoritmi di calcolo (`PercentileSketch`) e delle modalità di esecuzione.
- **Singleton:** `PerformanceMetrics` come punto unico e centralizzato di raccolta delle metriche di esecuzione.

# Query 1: Analisi Mensile Performance — RDD API

## Obiettivo

Aggregare i voli 2025 (AA vs DL) su base mensile. Calcolare min, max, media di *DEP\_DELAY* (solo voli non cancellati) e il *CANCELLATION RATE*.

### Paradigm 1: Low-Level RDD API

```
datasetVoli
  .mapToPair(v -> ([v.airline, v.month], v))
  .aggregateByKey(
    zeroValue: [sommaRitardi: 0, maxDelay: -inf, minDelay: +inf, nonCancellati: 0, totali: 0],
    combiner: (acc, v) -> {
      acc.totali += 1
      if !v.cancelled then
        acc.sommaRitardi += v.dep_delay
        acc.maxDelay = MAX(acc.maxDelay, v.dep_delay)
        acc.minDelay = MIN(acc.minDelay, v.dep_delay)
        acc.nonCancellati += 1
      return acc},
    reducer: (a1, a2) -> {
      a1.sommaRitardi += a2.sommaRitardi
      a1.maxDelay = MAX(a1.maxDelay, a2.maxDelay)
      a1.minDelay = MIN(a1.minDelay, a2.minDelay)
      a1.nonCancellati += a2.nonCancellati
      a1.totali += a2.totali
      return a1})
  .map(tuple -> {
    media = ROUND(tuple.sommaRitardi / tuple.nonCancellati, 2)
    tasso_canc = ROUND((tuple.totali - tuple.nonCancellati) / tuple.totali * 100, 2)
    return Row(tuple.month, tuple.airline, media, tuple.minDelay, tuple.maxDelay, tasso_canc)})
```

# Query 1: Analisi Mensile Performance — High-Level APIs

## Obiettivo

Aggregare i voli 2025 (AA vs DL) su base mensile. Calcolare min, max, media di *DEP\_DELAY* e il *CANCELLATION RATE*.

### Paradigm 2: DataFrame API (DSL)

```
datasetVoli
  .groupBy("MONTH", "OP_UNIQUE_CARRIER")
  .agg(
    avg(when(col("CANCELLED") == 0,
              col("DEP_DELAY"))).as("mean"),
    min(when(col("CANCELLED") == 0,
              col("DEP_DELAY"))).as("min"),
    max(when(col("CANCELLED") == 0,
              col("DEP_DELAY"))).as("max"),
    sum(when(col("CANCELLED") == 0,
              col("DEP_DELAY"))).as("sum")
  )
  .withColumnRenamed("MONTH", "month")
  .withColumnRenamed("OP_UNIQUE_CARRIER", "airline")
```

### Paradigm 3: Spark SQL

```
SELECT
  MONTH                AS month,
  OP_UNIQUE_CARRIER AS airline,

  AVG(CASE WHEN CANCELLED == 0
    THEN DEP_DELAY END) AS 'mean',
  MIN(CASE WHEN CANCELLED == 0
    THEN DEP_DELAY END) AS 'min',
  MAX(CASE WHEN CANCELLED == 0
    THEN DEP_DELAY END) AS 'max',

  (SUM(CANCELLED) / COUNT(*)) * 100 AS 'cancel_rate'

FROM flights
GROUP BY MONTH, OP_UNIQUE_CARRIER
```

## Query 2: Top 10 Ritardi e Scomposizione Cause — RDD API

### Obiettivo

Classifica prime 10 compagnie per *ARR\_DELAY* medio decrescente (voli non cancellati/deviati con conteggio  $\geq 500$ ), mostrando l'incidenza media di ciascuna causa di ritardo.

#### Paradigm 1: Low-Level RDD API

```
validFlights = flights.filter(f -> getSafe(f, CANCELLED) == 0 && getSafe(f, DIVERTED) == 0)
reducedRDD = validFlights
  .mapToPair(f -> (f.getString(OP_UNIQUE_CARRIER), f))
  .aggregateByKey(
    zeroValue: new double[7], // [0: Count, 1: Arr, 2: Carrier, 3: Weather, 4: NAS, 5: Security, 6: Late]
    combiner: (acc, f) -> {
      acc[0] += 1.0;
      acc[1] += getSafe(f, ARR_DELAY);
      acc[2] += getSafe(f, CARRIER_DELAY); acc[3] += getSafe(f, WEATHER_DELAY);
      acc[4] += getSafe(f, NAS_DELAY); acc[5] += getSafe(f, SECURITY_DELAY);
      acc[6] += getSafe(f, LATE_AIRCRAFT_DELAY);
      return acc;},
    reducer: (acc1, acc2) -> {
      for (int i=0; i<7; i++) acc1[i] += acc2[i];
      return acc1;})
processedRDD = reducedRDD
  .filter(t -> t._2[0] >= 500.0)
  .map(t -> {
    double c = t._2[0];
    return RowFactory.create(t._1, (long)c, t._2[1]/c, t._2[2]/c, t._2[3]/c, t._2[4]/c, t._2[5]/c, t._2[6]/c;)})
// Ordinamento globale ed estrazione Top 10 (Volume trascurabile post-filtraggio)
sortedRDD = processedRDD.sortBy(r -> r.getDouble(2), false, partitions)
top10RDD = sortedRDD.zipWithIndex().filter(t -> t._2 < 10).map(t -> t._1).coalesce(1)
```

## Query 2: Top 10 Ritardi e Scomposizione Cause — High-Level APIs

### Obiettivo

Classifica prime 10 compagnie per *ARR\_DELAY* medio decrescente (voli non cancellati/deviati con conteggio  $\geq 500$ ), mostrando l'incidenza media di ciascuna causa di ritardo.

#### Paradigm 2: DataFrame API (DSL)

```
flights
  .filter(col("CANCELLED").equalTo(0)
    .and(col("DIVERTED").equalTo(0)))
  .groupBy("OP_UNIQUE_CARRIER")
  .agg(
    count("*").as("num_flights"),
    avg(coalesce(col("ARR_DELAY"),
      lit(0))).as("arrdelay_mean"),
    avg(coalesce(col("CARRIER_DELAY"),
      lit(0))).as("carrier_mean"),
    avg(coalesce(col("WEATHER_DELAY"),
      lit(0))).as("weather_mean"),
    avg(coalesce(col("NAS_DELAY"),
      lit(0))).as("nas_mean"),
    avg(coalesce(col("SECURITY_DELAY"),
      lit(0))).as("security_mean"),
    avg(coalesce(col("LATE_AIRCRAFT_DELAY"),
      lit(0))).as("late_aircraft_mean")
  )
  .withColumnRenamed("OP_UNIQUE_CARRIER", "carrier")
  .filter(col("num_flights").geq(500))
  .orderBy(col("arrdelay_mean").desc())
  .limit(10)
```

#### Paradigm 3: Spark SQL

```
SELECT
  OP_UNIQUE_CARRIER AS carrier,
  COUNT(*)           AS num_flights,
  AVG(COALESCE(ARR_DELAY, 0)) AS arrdelay_mean,
  AVG(COALESCE(CARRIER_DELAY, 0)) AS
    carrier_delay_mean,
  AVG(COALESCE(WEATHER_DELAY, 0)) AS
    weather_delay_mean,
  AVG(COALESCE(NAS_DELAY, 0)) AS nas_delay_mean,
  AVG(COALESCE(SECURITY_DELAY, 0)) AS
    security_delay_mean,
  AVG(COALESCE(LATE_AIRCRAFT_DELAY, 0)) AS
    late_aircraft_delay_mean
FROM flights
WHERE CANCELLED = 0 AND DIVERTED = 0
GROUP BY OP_UNIQUE_CARRIER
HAVING num_flights >= 500
ORDER BY arrdelay_mean DESC
LIMIT 10
```

## Query 3: Percentili Orari e Min/Max Globali — RDD API

### Obiettivo

Calcolo dei percentili (25, 50, 75, 90) del *DEP\_DELAY* per fascia oraria (24h) tramite algoritmi di sketch (*KLL/t-digest*) e calcolo parallelo di min/max globali.

### Paradigm 1: RDD API (Dual-Pipeline Architecture)

```
// Subset condiviso e mantenuto in memoria per evitare scansioni multiple
validFlights = flights.filter(f -> f.getDouble(CANCELLED) == 0.0 && !f.isNullAt(DEP_DELAY)).cache()
// Pipeline 1: Calcolo Percentili Orari tramite Sketch distribuiti
PercentileSketch sketch = PercentileSketch.from(config.getPercentileAlgorithm())
hourlyRows = validFlights
    .filter(f -> !f.isNullAt(CRS_DEP_TIME))
    .mapToPair(f -> {
        int hour = (f.getInt(CRS_DEP_TIME) / 100) % 24;
        return new Tuple2<>(new Tuple2<>(f.getString(CARRIER), hour), f.getDouble(DEP_DELAY));})
    .combineByKey(sketch::init, sketch::update, sketch::merge)
    .map(t -> {
        double[] q = sketch.getQuantiles(t._2, 0.25, 0.50, 0.75, 0.90);
        return RowFactory.create(t._1._1, t._1._2, round2(q[0]), round2(q[1]), round2(q[2]), round2(q[3]));})
// Pipeline 2: Aggregazione Globale Min/Max per Compagnia
globalRows = validFlights
    .mapToPair(f -> new Tuple2<>(f.getString(CARRIER), f))
    .aggregateByKey(
        zeroValue: new double[]{Double.MAX_VALUE, -Double.MAX_VALUE},
        combiner: (acc, f) -> {
            double d = f.getDouble(DEP_DELAY);
            acc[0] = Math.min(acc[0], d); acc[1] = Math.max(acc[1], d);
            return acc;
        },
        reducer: (a1, a2) -> { a1[0] = Math.min(a1[0], a2[0]); a1[1] = Math.max(a1[1], a2[1]); return a1;})
    .map(t -> RowFactory.create(t._1, t._2[0], t._2[1]))
```

## Query 3: Percentili Orari e Min/Max Globali — High-Level APIs

### Obiettivo

Calcolo dei percentili (25, 50, 75, 90) del *DEP\_DELAY* per fascia oraria (24h) tramite algoritmi di sketch integrati (*KLL/t-digest*) e calcolo parallelo di min/max globali.

#### Paradigm 2: DataFrame API (DSL)

```
flights.cache()
// Pipeline 1: Percentili orari approssimati
hourlyStats = flights
  .filter(col("CANCELLED").equalTo(0))
  .withColumn("hour",
    col("CRS_DEP_TIME").divide(100).cast("int"))
  .groupBy("OP_UNIQUE_CARRIER", "hour")
  .agg(
    expr("percentile_approx(DEP_DELAY,
      0.25)").as("p25"),
    expr("percentile_approx(DEP_DELAY,
      0.50)").as("p50"),
    expr("percentile_approx(DEP_DELAY,
      0.75)").as("p75"),
    expr("percentile_approx(DEP_DELAY,
      0.90)").as("p90"))
  .withColumnRenamed("OP_UNIQUE_CARRIER", "airline")
// Pipeline 2: Min/Max assoluti
globalMinMax = flights
  .groupBy("OP_UNIQUE_CARRIER")
  .agg(
    min("DEP_DELAY").as("min_delay"),
    max("DEP_DELAY").as("max_delay"))
  .withColumnRenamed("OP_UNIQUE_CARRIER", "airline")
```

#### Paradigm 3: Spark SQL

```
-- Pipeline 1: Stima quantili orari
SELECT
  OP_UNIQUE_CARRIER AS airline,
  CAST(CRS_DEP_TIME / 100 AS INT) AS hour,
  percentile_approx(DEP_DELAY, 0.25) AS p25,
  percentile_approx(DEP_DELAY, 0.50) AS p50,
  percentile_approx(DEP_DELAY, 0.75) AS p75,
  percentile_approx(DEP_DELAY, 0.90) AS p90
FROM flights
WHERE CANCELLED = 0
GROUP BY OP_UNIQUE_CARRIER, hour;

-- Pipeline 2: Estremi globali
SELECT
  OP_UNIQUE_CARRIER AS airline,
  MIN(DEP_DELAY) AS min_delay,
  MAX(DEP_DELAY) AS max_delay
FROM flights
GROUP BY OP_UNIQUE_CARRIER;
```

# Algoritmi Percentili a Confronto: T-Digest vs KLL Sketch

## T-Digest (Ted Dunning's Library)

```
// Inizializzazione buffer
MergingDigest digest = new MergingDigest(100.0);

// Update / Merge via ByteBuffer
MergingDigest.fromBytes(ByteBuffer.wrap(state));

// Compattazione in byte array
ByteBuffer buf = ByteBuffer.allocate(
    digest.smallByteSize()
);
digest.asSmallBytes(buf);
return buf.array();
```

- **Accuratezza:**  $\sim 1\%$  errore nelle code.
- **Parametro:** COMPRESSION = 100.0.
- $\rightarrow$  Ottimo per l'analisi accurata degli estremi (es. ritardi record).

## KLL Sketch (Apache DataSketches)

```
// Allocazione nativa in Heap
KllDoublesSketch.newHeapInstance();

// Ricostruzione stato tramite Memory
KllDoublesSketch.heapify(Memory.wrap(state));

// Export diretto nativo ultra-veloce
return sketch.toByteArray();

// Estrazione quantili con criterio inclusivo
sketch.getQuantile(q,
    QuantileSearchCriteria.INCLUSIVE);
```

- **Accuratezza:** Errore di rango  $\sim 1.65\%$ .
- **Parametro:** Default k=200 (ottimale).
- $\rightarrow$  Più veloce nell'I/O grazie all'integrazione con lo strato Memory.



## Query 4: Airline Clustering — ML Pipeline

### Obiettivo Analisi

Individuare gruppi di vettori con comportamenti simili su 11 metriche aggregate (11D), standardizzando lo spazio multivariato ed applicando l'algoritmo K-Means.

#### Silhouette Analysis & K-Means

```
ClusteringEvaluator evaluator = new
    ClusteringEvaluator()
    .setFeaturesCol("features")
    .setPredictionCol("prediction")
    .setMetricName("silhouette");

int bestK = 2; double bestScore = -1.0;
for (int k = 2; k <= 5; k++) {
    KMeans km = new KMeans().setK(k)
        .setSeed(42L).setInitMode("k-means||");
    KMeansModel m = km.fit(scaledData);
    double score =
        evaluator.evaluate(m.transform(scaledData));
    if (score > bestScore) {
        bestScore = score; bestK = k; }
}

KMeansModel modelFinal = new KMeans()
    .setK(bestK).setSeed(42L).fit(scaledData);
```

- **Scaling:** Spazio standardizzato a 11D tramite StandardScaler.
- **Ottimizzazione:** Selezione iterativa del  $K$  per ridurre l'arbitrarietà.

#### Proiezione PCA per Visualizzazione

```
PCA pca = new PCA()
    .setInputCol("features")
    .setOutputCol("pca_features")
    .setK(2);

PCAModel pcaModel = pca.fit(finalPredictions);

// Espansione del vettore nei componenti x e y
Dataset<Row> finalOutput = appendPcaColumns(
    pcaModel.transform(finalPredictions)
);
```

- **Dimensionalità:** Il clustering avviene nel blocco nativo a 11D.
- **Riduzione Geometrica:** PCA applicata *post-clustering* solo per mappare i punti sul piano cartesiano 2D.

# Spark: Misurazione delle Metriche

## Event-Driven Listening

- JobTimerListener estende SparkListener.
- Intercettazione asincrona degli eventi del ciclo di esecuzione di Spark.

## Tracciamento Granulare

- **Job e Stage Level:** Isolamento delle durate pure dei singoli stage e job.
- **I/O e Network Balance:** Misurazione sia dei dati di I/O che dei dati scambiati durante le fasi di Shuffle.
- **Worker Profiling:** Misurazione per singolo Executor per individuare fenomeni di Data Skew.

### JobTimerListener

```
@Override
public void
    onStageCompleted(SparkListenerStageCompleted sc)
{
    TaskMetrics tm = sc.stageInfo().taskMetrics();
    long duration = System.currentTimeMillis() - start;
    long gcTime = tm.jvmGCTime();
    long bytesRead = tm.inputMetrics().bytesRead();
    long shuffleRead =
        tm.shuffleReadMetrics().totalBytesRead();
    ...

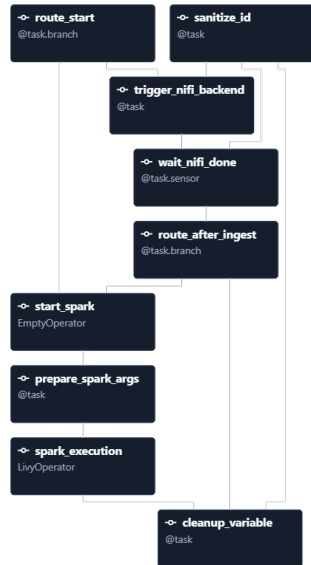
    PerformanceMetrics.getInstance()
        .addStageMetrics(...);
}

@Override
public void onTaskEnd(SparkListenerTaskEnd te) {
    String executorId = te.taskInfo().executorId();
    TaskMetrics tm = te.stageInfo().taskMetrics();
    long bytesRead = tm.inputMetrics().bytesRead();
    long serializeTime = tm.resultSerializationTime();
    ...

    PerformanceMetrics.getInstance()
        .addExecutorMetrics(...);
}
```

# Orchestrazione End-to-End: flight\_analysis

- **Obiettivo del Workflow:** Eseguire l'intera pipeline, dalla fase di Ingestion a quella di Processamento.
- **Workflow Operativo:**
  - `route_start`: Valutazione della modalità operativa per il branching del flusso: *Ingest*, *Execution*, *Both*.
  - `trigger_nifi_backend`: Generazione del payload JSON e iniezione dei metadati via chiamata REST a NiFi.
  - `wait_nifi_done`: Sincronizzazione asincrona tramite Sensor (polling su variabile aggiornata da NiFi via callback).
  - `prepare_spark_args`: Mapping dei parametri del DAG in argomenti CLI per il job Spark.
  - `spark_execution`: Sottomissione del job Spark tramite LivyOperator in modalità client.



# Orchestrazione del Benchmarking: EC2 vs EMR

- **Obiettivo:** Valutazione sistematica di tutte le combinazioni sugli ambienti di deployment.
- Creazione dinamica di una pipeline sequenziale, garantendo l'isolamento delle risorse per ogni job Spark.

## Cluster Self-Hosted

```
spark_task = LivyOperator(  
    task_id=task_id,  
    livy_conn_id="livy_default",  
    file=JAR_PATH,  
    class_name=JAR_CLASS,  
    args=[  
        "--query", query,  
        "--backend", backend,  
        "--config", "{{ params.config_file }}",  
        "--input-type", "{{ params.input_type }}",  
        "--output-type", "{{ params.output_type }}",  
        "--metrics-type", "{{ params.metrics_type }}"  
    ],  
    name=f"benchmark-{query}-{backend}",  
    polling_interval=5  
)
```

## AWS EMR

```
spark_step = {  
    "Name": step_name,  
    "ActionOnFailure": "CONTINUE",  
    "HadoopJarStep": {  
        "Jar": "command-runner.jar",  
        "Args": [  
            "spark-submit", "--deploy-mode", "client",  
            "--class", JAR_CLASS, JAR_PATH,  
            "--query", query, ...  
        ],  
    },  
}  
  
add_step = EmrAddStepsOperator(  
    task_id=tid,  
    job_flow_id="{{ params.job_flow_id }}",  
    aws_conn_id="aws_default",  
    steps=[spark_step]  
)  
  
watch_step = EmrStepSensor(  
    task_id=task_id,  
    job_flow_id="{{ params.job_flow_id }}",  
    step_id=f"{{{ ti.xcom_pull(task_ids='{tid}')[0] }}}",  
    aws_conn_id="aws_default",  
    poke_interval=30  
)
```

- **Rigore Statistico:** Ogni combinazione (Query  $\times$  Backend) è stata eseguita **5 volte** per garantire la stabilità delle misurazioni.
- **Telemetria:** Utilizzo del JobTimerListener (SparkListener) per isolare il *tempo netto di calcolo* dall'overhead di sistema.
- **Hardware e Infrastruttura:**
  - **EC2 (Self-Hosted):** Master + Worker su istanze t3.small. Storage basato su HDFS.
  - **EMR (Cloud-Native):** 1 Master + 3 Core su istanze m6g.xlarge (ARM64). Architettura *Shared-Nothing* su Amazon S3 via s3a://.

# Confronto API: RDD vs DataFrame vs SQL

## Aggregazioni Massive: Q1 e Q2

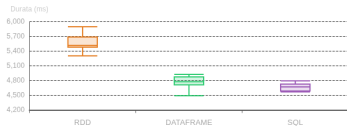
- DataFrame e SQL mostrano mediane temporali inferiori e una dispersione contratta in ogni ambiente.

Spark Execution Time Distribution



Distribuzione tempi Q1 con partition pruning su EMR

Spark Execution Time Distribution

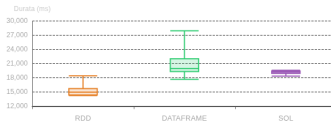


Distribuzione tempi Q2 con partition pruning su EMR

## Approssimazione Percentili: Q3

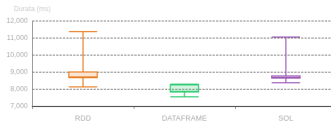
- **Self-Hosted EC2:** RDD con KLL risulta mediamente più veloce.
- **Managed su EMR:** EMR azzerava il vantaggio di RDD e DataFrame riprende il primato.

Spark Execution Time Distribution



Distribuzione tempi Q3 con partition pruning e KLL su EC2

Spark Execution Time Distribution



Distribuzione tempi Q3 con partition pruning e KLL su EMR

# Ottimizzazione I/O: Pruning vs Pushdown

- **Predicate Pushdown:** Genera un volume di lettura superiore e tende a sovraccaricare singoli nodi.



Sbilanciamento I/O con predicate pushdown su EC2 nella Q1



Sbilanciamento I/O con predicate pushdown su EC2 nella Q2

- **Partition Pruning:** Esclude i file a livello di file system, diminuendo il *Read Skew* fra i nodi.



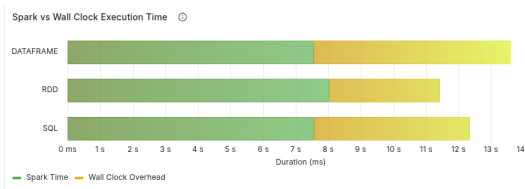
Sbilanciamento I/O con partition pruning su EC2 nella Q1



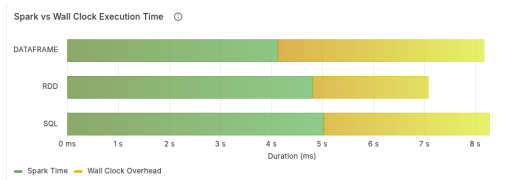
Sbilanciamento I/O con partition pruning su EC2 nella Q2

# Overhead Infrastrutturale: EC2 vs EMR

- EMR riduce sensibilmente l'overhead di sistema, con un rapporto di *Execution Time* vs *Wall-Clock Time* più favorevole rispetto a EC2.



Overhead di sistema con predicate pushdown su EC2 nella Q1



Overhead di sistema con predicate pushdown su EMR nella Q1

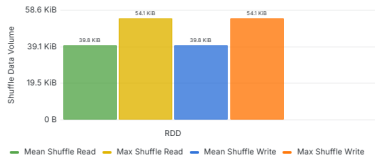


# Confronto Sketch: KLL vs T-Digest

## KLL Sketch

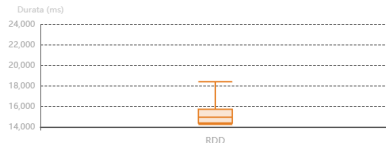
- Richiede un maggiore impiego di memoria per preservare la struttura a piramide dei livelli.

Shuffle Intensity: Internal Data Exchange ⓘ



Volume di Shuffle Read su EC2 per KLL con partition pruning

Spark Execution Time Distribution ⓘ

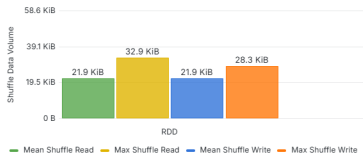


Tempo di esecuzione su EC2 per KLL con partition pruning

## T-Digest

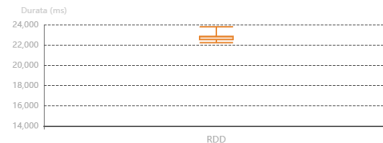
- Altamente compatto in termini di memoria, poiché memorizza unicamente i centroidi.

Shuffle Intensity: Internal Data Exchange ⓘ



Volume di Shuffle Read su EC2 per T-Digest con partition pruning

Spark Execution Time Distribution ⓘ



Tempo di esecuzione su EC2 per T-Digest con partition pruning

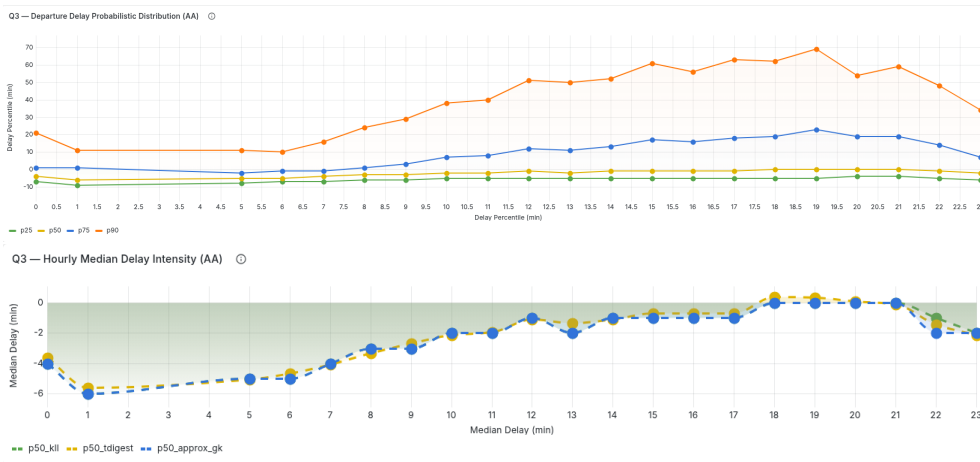
# Risultati Analitici: Q3 - Percentili Orari

## Dinamiche Operative

- Le performance degradano sistematicamente nelle fasce serali probabilmente a causa della propagazione a catena dei ritardi.

## Confronto Algoritmico

- Convergenza analitica dei risultati tra le euristiche.
- La selezione dipende dai vincoli non funzionali.



# Risultati Analitici: Q4 - Airline Clustering

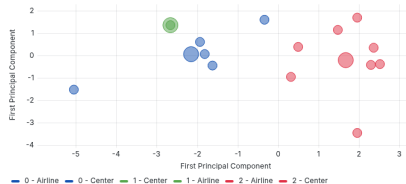
## Profilazione delle Compagnie

- **C0 - Inefficienza:** AA, B6, 00, F9, OH. Forte propagazione a catena dei ritardi e picchi di cancellazioni ( $> 6\%$ ).
- **C1 - Resilienza:** G4. Forti ritardi per cause esterne, ma altissimo completamento voli (canc. 0.6%).
- **C2 - Virtuosi:** UA, NK, DL, MQ, HA, YX, AS, WS. Stabilità massima e forte capacità di recupero tempo in volo.

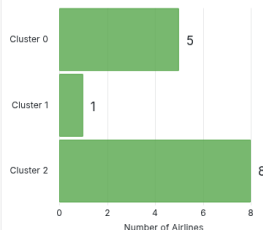
Q4 — Multi-dimensional Centroid Profiles



Q4 — Latent Performance Clusters (PCA Projection)



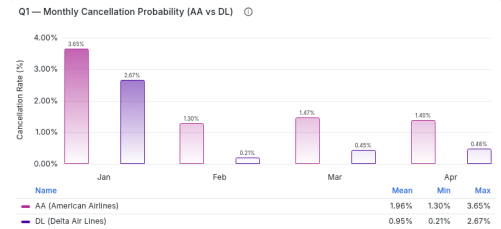
Q4 — Population Distribution by Cluster



# Risultati Analitici: Q1 e Q2 - Performance e Cause

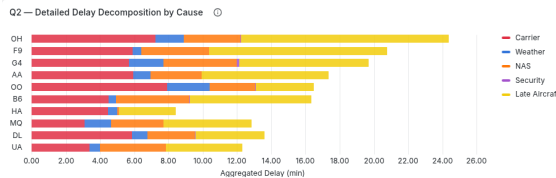
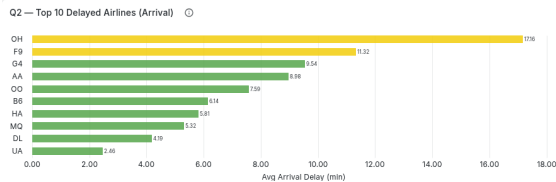
## Q1: Performance Mensili

- DL:** Miglioramento costante. Il ritardo scende e le cancellazioni si stabilizzano sotto lo 0.5%.
- AA:** Degrado progressivo. Il ritardo sale con un tasso di cancellazione ~ 1.4%.



## Q2: Decomposizione dei Ritardi

- OH:** Peggior vettore per ritardo in arrivo. Forte incapacità di assorbire i ritardi pregressi (*Late Aircraft delay*).
- UA:** Altamente virtuoso, ma fortemente esposto alla congestione del traffico aereo (*NAS delay*).



## Conclusioni

- **Ecosistema Enterprise-Ready:** L'orchestrazione asincrona con Airflow combinata alla modularità di Spark e NiFi garantisce scalabilità e robustezza.
- **Ottimizzazione dell'I/O:** L'uso congiunto del formato colonnare Parquet e del Partition Pruning diminuisce l'overhead di lettura sul file system distribuito.
- **Analitica Avanzata e Sostenibile:** L'integrazione di Quantile Sketches e Model Selection automatizzato decodifica dinamiche complesse mantenendo costi computazionali accettabili.

## Sviluppi Futuri

- **Architettura Streaming:** Integrazione di Apache Kafka e Spark Structured Streaming per l'acquisizione della telemetria di volo in tempo reale.
- **Predictive Analytics on-the-fly:** Sfruttare i centroidi dei modelli ML pre-addestrati per classificare la stabilità della rete e prevedere anomalie in tempo reale.

# Grazie per l'attenzione



**TOR VERGATA**  
UNIVERSITÀ DEGLI STUDI DI ROMA